Probabilistic Forecasting in the JAX Ecosystem: NumPyro, Composability, and Community

Juan Orduz[a,*], Du Phan[1], Theo Rashid[b]

[a] *PyMC-Labs*
[b] *Amazon*

Abstract

Open-source forecasting is often framed as a competition between turn-key libraries that wrap canonical models behind friendly APIs. This paper argues for a complementary mode: composable probabilistic substrates such as NumPyro, layered on the JAX ecosystem, where practitioners assemble bespoke models from a small set of primitives, supported by a community layer of tutorials and worked examples. We trace the Pyro lineage NumPyro inherits, isolate the two primitives that matter most for time series, review inference tooling that composes with the wider JAX stack, and single out structured covariate handling as the capability that most often justifies a substrate over a packaged forecaster. Six case studies exercise this flexibility across hierarchical pooling, intermittent and censored demand, Bayesian vector autoregression, Gaussian processes, and prior calibration. We discuss when a substrate complements pre-packaged forecasters, the steep learning curve and how the community lowers it, and open problems at the software–forecasting intersection.

Keywords: Probabilistic forecasting, Bayesian methods, Open-source software, Hierarchical models, Intermittent demand, Gaussian processes

1. Introduction

Forecasting drives consequential decisions across retail, energy, finance, public health, and operations research. The decisions that follow a forecast are rarely symmetric in their costs: a stockout differs from an overstock, an under-provisioned grid differs from an over-provisioned one, and a missed marketing window differs from an inefficient one. Point forecasts collapse this asymmetry

*Corresponding author.
Email addresses: juan.orduz@pymc-labs.com (Juan Orduz), fehiepsi@gmail.com (Du Phan)
[1] Currently at Google DeepMind.

into a single number and force the consumer of the forecast to invent uncertainty after the fact. Probabilistic forecasts, by contrast, carry the uncertainty alongside the prediction and let downstream decision makers reason about quantiles, tails, and joint events in a principled way, with evaluation through proper scoring rules such as the continuous ranked probability score (Gneiting & Raftery, 2007; Gneiting & Katzfuss, 2014). A further distinction matters here. Rather than merely a collection of marginal quantiles, a structural Bayesian model yields coherent joint draws from the data-generating process. Sampling whole trajectories exposes scenarios and correlated tail events that median-centered intervals hide. In election forecasting, for instance, individual draws represent realistic scenarios, maintaining correlations in voting patterns between subnational populations that marginal summaries would obscure (Heidemanns, Gelman & Morris, 2020).

The open-source community has responded to this need with an impressive landscape of forecasting tools. Pre-packaged libraries such as the Nixtla ecosystem and sktime expose canonical statistical and machine-learning methods through friendly user-facing APIs (Nixtla, 2024; Löning, Bagnall, Ganesh, Kazakov, Lines & Király, 2019). GluonTS bundles deep probabilistic forecasters with a consistent training and evaluation harness (Alexandrov, Benidis, Bohlke-Schneider et al., 2020). More recently, large pre-trained time-series foundation models such as Chronos and TimesFM offer zero-shot forecasts behind comparably simple interfaces (Ansari, Stella et al., 2024; Das, Kong, Sen & Zhou, 2023). The classical statistical modeling tradition is well served by statsmodels (Seabold & Perktold, 2010) and by R libraries that have shaped a generation of forecasting practice (Hyndman & Athanasopoulos, 2021). These libraries succeed precisely because they hide model implementation behind a uniform interface and let practitioners focus on data, validation, and deployment.

This paper is concerned with a different layer of the open-source forecasting stack. There is a recurring class of forecasting problems where the canonical model menu is the wrong starting point. Demand series may contain zeros that mix true zero demand with stockout-censored observations. A retailer may want to share information across thousands of related series in ways that respect both group-level and unit-level idiosyncrasies. A practitioner may need full control over how covariates enter the model, encoding holiday and event effects with bespoke shapes, including before-and-after windows, Fourier terms, or bump functions, rather than a single indicator column, while a standard time series component handles the business-as-usual dynamics. An analyst may want to combine a long historical time series with a small number of expert-elicited prior anchors or with

auxiliary measurements from a designed experiment. In each case the modeling work is not picking the right item from a menu. It is composing a custom probabilistic model from smaller building blocks, fitting it with the most appropriate inference algorithm, and connecting it to the rest of a JAX-based analytical pipeline.

NumPyro (Phan, Pradhan & Jankowiak, 2019) occupies this layer. It is a lightweight probabilistic programming substrate built on JAX (Bradbury, Frostig, Hawkins, Johnson, Leary, Maclaurin, Necula, Paszke, VanderPlas, Wanderman-Milne & Zhang, 2018), inheriting the design philosophy of Pyro (Bingham, Chen, Jankowiak, Obermeyer, Pradhan, Karaletsos, Singh, Szerlip, Horsfall & Goodman, 2019) while compiling and accelerating inference through the same compositional transformations that power the rest of the JAX ecosystem. Its primitives are deliberately spare. A small number of them, together with the underlying JAX transforms, are sufficient to express most of the recurring forecasting patterns encountered in practice. The cost is a steep learning curve, since users are exposed to Bayesian modeling, JAX semantics, and a particular control-flow style at the same time. The mitigation is community: a growing body of worked examples, tutorials, and blog posts that progressively close the gap between framework primitives and end-to-end forecasting workflows.

The contributions are threefold. First, we situate NumPyro in the wider open-source forecasting landscape and explain when a composable substrate is the right answer and when a pre-packaged library is. Second, we present the small set of NumPyro primitives and inference tools that matter for time series, showing how forecasting demands shape the design philosophy and JAX-ecosystem composability of a substrate, and we single out structured covariate handling as the capability that most often makes the substrate worthwhile. Third, we survey six publicly available case studies and treat community content as a first-class part of the open-source forecasting story. Our emphasis is the intersection of software and forecasting: how forecasting problems inform the abstractions a substrate exposes.

The remainder of the paper is organized as follows. Section 2 traces the Pyro lineage NumPyro builds upon. Section 3 introduces the relevant software primitives and inference tools. Section 4 isolates structured covariate handling as the capability that most often justifies the substrate. Section 5 surveys six forecasting case studies. Section 6 reflects on the community layer that supports the substrate. Section 7 discusses the trade-offs, limitations, and open problems. Section 8

concludes.

2. Standing on the Shoulders of Giants

This section is gratitude, not comparison. NumPyro is best understood as the JAX-native continuation of an idea developed by the Pyro project at Uber AI Labs. The broader argument for composable probabilistic substrates applies to the class of tools that let practitioners write models in a host language rather than a separate modeling DSL, including Stan, PyMC, TensorFlow Probability, and Turing.jl (several of which ship dedicated state-space or structural time-series modules, such as the `statespace` module in PyMC-Extras and `tfp.sts` in TensorFlow Probability (PyMC Developers, 2024; TensorFlow Probability Development Team, 2024)); we study NumPyro because its JAX foundation makes ecosystem composition the central design payoff. What distinguishes the Pyro/NumPyro lineage is a design choice rather than a feature list: a model is an ordinary Python function written against an array library, so the model reads close to its mathematical statement.

Pyro (Bingham et al., 2019) introduced a programming model around two key ideas: probabilistic models as ordinary Python functions with `sample` statements, and the *effect handler* abstraction that separates defining a generative model from interpreting it. Effect handlers are the high-level abstraction that makes this separation possible: a handler is an enclosing context that intercepts the primitive statements a model executes (`sample`, `param`, `plate`) and reinterprets their effect, without the model body being aware of it or rewritten. Because the model is just a function emitting these primitives, the same function can be traced to record its execution, conditioned on observed data, replayed against a guide, or retargeted to an arbitrary inference strategy simply by running it under a different stack of handlers (NumPyro Development Team, 2023a). This is what lets one model definition serve prior sampling, posterior inference, and predictive simulation interchangeably.

The forecasting community owes Pyro a more specific debt. `pyro.contrib.forecast` distilled state-space forecasting into reusable infrastructure, and `pyro.contrib.epidemiology` applied the same `plate`-heavy machinery to epidemiological models at scale (Obermeyer, Jankowiak et al., 2022); that forecasting infrastructure has recently been ported to the JAX era as `numpyro_forecast` (Orduz, 2026c). The port pairs a pure functional core with an interchangeable object-oriented interface carried over from Pyro (`ForecastingModel`, with `Forecaster` for variational inference and `HMCForecaster` for Hamiltonian Monte Carlo), ships time-series cross-validation through rolling-

and expanding-window backtesting, including a vectorized variant that fits every window in a single `vmap`-batched SVI run over a stack of cutoff dates, and implements point and probabilistic evaluation metrics such as the empirical continuous ranked probability score and empirical interval coverage. The Pyro M5 Starter Kit (Pyro Development Team, 2020b) showed the primitives scaling to thousands of related series. The roles of `scan` and `plate` in this paper were sharpened within Pyro before being inherited by NumPyro.

NumPyro (Phan et al., 2019) reimplements the same conceptual model on top of JAX. Models participate in the full transformation stack (`jit`, `vmap`, `grad`), inference reuses JAX-native NUTS and SVI (Hoffman & Gelman, 2014), and the optimizer interface plugs into Optax (DeepMind, Babuschkin, Baumli, Bell et al., 2020b) rather than reimplementing the full stack internally. Because JAX programs are pure functions, an entire inference routine compiles to a single fused program rather than executing eagerly operation by operation, which is the main practical gain over Pyro-on-PyTorch. The same property lets NumPyro plug into the surrounding ecosystem: `dynestyx` for dynamical systems (Basis Research, 2024), Flax for specifying neural networks (Heek, Levskaya, Oliver, Ritter et al., 2020), and a broad catalog of compatible tools (Vadivelu et al., 2024; DeepMind, Babuschkin, Baumli, Bell et al., 2020a). The epidemiology thread that ran through Pyro continues in the JAX era: PyRenew, a Bayesian renewal-modeling package for infectious-disease forecasting, is built directly on NumPyro (CDC Center for Forecasting and Outbreak Analytics, 2024), evidence that the substrate is solid enough to carry domain-specific forecasting libraries downstream. Custom intermittent-demand transitions, hierarchical priors, additional likelihoods, and Hilbert-space Gaussian processes all become natural to express because the substrate was designed to be composable rather than complete.

3. A Compact Software Ecosystem for Probabilistic Forecasting

A common worry about probabilistic programming is that productive use requires fluency in a sprawling API. Our experience writing the worked examples surveyed in Section 5 suggests the opposite: a small number of primitives, together with two inference strategies and a few JAX transforms, cover most forecasting needs. This section names the pieces.

3.1. Two primitives that matter for time series

`scan`. Many time series models are defined recursively. An exponential smoothing state evolves from the previous state. A linear Gaussian state-space model passes a latent process through a transition function and an observation function. A vector autoregression updates a multivariate state by a linear map plus innovation noise. Intermittent-demand models such as Croston and Teunter-Syntetos-Babai (TSB) update a level and an interval (or probability) component on each step (Croston, 1972; Teunter, Syntetos & Babai, 2011). Even models not usually written as recursions, from moving-average processes to multi-horizon and attention-based forecasters, can be cast in this form.

JAX exposes recursion through `jax.lax.scan`, and NumPyro provides a probabilistic counterpart that lets each step contain `sample` statements without breaking JIT compilation or gradient propagation. Operationally, `scan` is a compiled `for` loop with a fixed-shape carry. Almost every forecasting model in this paper is a single `scan` over a transition function (Orduz, 2023e,d,b). The case studies in Sections 5.2 and 5.3 illustrate a further payoff: the practitioner owns the transition body and can add conditional branches for operational rules while keeping the same `scan` skeleton.

This makes classical state-space models strikingly easy to write. A full linear-Gaussian state-space model is nothing more than a transition function and an observation function wrapped in a single `scan`: a local linear trend model, for instance, is one `scan` whose transition advances level and slope and emits one observation per step, where the body is an ordinary state update and `scan` supplies the time recursion (Orduz, 2026a). There is no separate filtering machinery to invoke and no special-case API; the worked exponential-smoothing-in-state-space-form example shows the entire model in a few lines (Orduz, 2026a; NumPyro Development Team, 2023c). The Bayesian VAR of Section 5.4 is the same idea carried to a multivariate state. The flip side is computational: `scan` steps through time sequentially and samples the latent states rather than marginalizing them, so when a model is linear-Gaussian a dedicated Kalman filter can be the more efficient inference route, a trade-off we return to when discussing ecosystem composition.

`plate`. Forecasting work is almost never single-series. Retailers track tens of thousands of stock-keeping units. Energy utilities forecast hundreds of substations. Transit agencies forecast every station-to-station flow. The probabilistic programming convention for expressing that a set of random variables are conditionally independent across an indexed dimension, within the joint Bayesian

model, is the `plate` construct, which originates from graphical-model notation. The qualifier matters: the units are conditionally independent given shared population-level parameters, so they remain coupled through those hyperpriors and are fit jointly rather than as separate models, which is exactly what lets a short series borrow strength from the rest of the panel. In NumPyro a `plate` both annotates conditional independence and triggers efficient vectorized sampling along the corresponding axis. Hierarchical models, panel models, and any batched evaluation use `plate` to push parallelism down into the JAX compiler.

The combination of `scan` and `plate` is what makes hierarchical state-space models practical. A typical pattern is a `plate` over the unit dimension that wraps a `scan` over time. The compiler sees a doubly batched computation and produces highly efficient device code regardless of whether the model has a hundred series or a hundred thousand. The hierarchical forecasting blog series (Orduz, 2024d,e,f) is built almost entirely on this pattern.

Concretely, the pattern is a population-level prior, a `plate` over series, and a `scan` over time nested inside it (NumPyro Development Team, 2023b); the JAX compiler fuses the two batch dimensions into a single program.

We deliberately stop the primitives tour here. Other NumPyro primitives exist and appear naturally inside specific models, but a working knowledge of `scan` and `plate` is sufficient to follow every model in this paper.

3.2. Inference, where the ecosystem shows

Markov chain Monte Carlo with NUTS.. For models with up to a few thousand parameters, the default inference recipe is the No-U-Turn Sampler (Hoffman & Gelman, 2014), accessed via `numpyro.infer.MCMC` with a `NUTS` kernel. NumPyro's NUTS is a JIT-compiled JAX program; by default multiple chains run in parallel across devices via `pmap` (the `chain_method="parallel"` setting), with a `vectorized` option that batches chains on a single device through `vmap`. This makes warmup and sampling considerably faster than reference implementations in less compilation-friendly frameworks. The Bayesian VAR example (Orduz, 2023b) uses NUTS to obtain full joint posteriors over autoregressive coefficients and an LKJ-distributed innovation covariance.

Stochastic variational inference.. When the model is too large for full MCMC, or when only marginal posteriors and approximate posterior predictives are needed, stochastic variational inference (Blei, Kucukelbir & McAuliffe, 2017) is the workhorse. NumPyro provides `numpyro.infer.SVI`

together with a family of automatic guides, including `AutoNormal`, `AutoMultivariateNormal`, and `AutoLowRankMultivariateNormal`, that construct a tractable variational family from the model structure. Practitioners can also write custom guides when domain knowledge suggests a richer approximation. The blog post on SVI with NumPyro (Orduz, 2024h) walks through the workflow end-to-end.

Composing with Optax.. NumPyro ships only thin built-in optimizer wrappers and accepts any Optax optimizer directly (DeepMind et al., 2020b), so gradient clipping, learning-rate warmup, and AdamW recipes from deep learning work unmodified for variational inference. Each library does one thing, and they fit together because their authors agreed on shared abstractions.

Posterior predictive sampling.. Once inference returns a posterior or a variational approximation, predictive distributions are generated through `numpyro.infer.Predictive`. Posterior predictive sampling is JIT-compilable and vectorizable, so producing thousands of forecast trajectories for thousands of series is a single batched call rather than a Python loop.

3.3. Composing with the wider JAX ecosystem

The composability we keep invoking is concrete, not rhetorical. Because a NumPyro model is a pure JAX function, it exposes a log-density that external inference engines can target directly, so the same model can be handed to other JAX inference libraries without being rewritten. BlackJAX provides a modular collection of samplers and variational methods (Cabezas, Corenflos, Lao, Louf et al., 2024), and flowMC adds normalizing-flow enhanced Markov chain Monte Carlo (Wong, Gabrié & Foreman-Mackey, 2023); both can run inference on a NumPyro model, which matters when a particularly difficult posterior geometry calls for samplers beyond the built-in menu; the NumPyro documentation includes a worked example that does exactly this (NumPyro Development Team, 2023d). A third form of composition concerns state-space inference itself. A sequential `scan` is one way to evaluate a recursive model, but when a component has linear-Gaussian structure, Kalman filter techniques marginalize latent states analytically and admit parallel-in-time implementations, so the time dimension can be parallelized on accelerators rather than stepped through sequentially. `cuthbert` (Duffield, Iqbal & Corenflos, 2026) decouples model specification from inference and provides filtering, smoothing, and parameter estimation; `dynestyx` (Basis Research, 2024) specifies a state-space model as a NumPyro model and calls `cuthbert` for the filtering. The substrate

therefore supports both sequential recursion when the model demands it and specialized state-space inference when the structure allows it.

Neural networks compose equally directly: the M4-winning exponential-smoothing RNN is a structural model with neural components estimated end-to-end (Smyl, 2020), and a Flax module can be embedded inside a NumPyro model (Orduz, 2024f). Composition with large pre-trained forecasters is harder: openly released model checkpoints make a frozen forecaster callable from the same pipeline (Ansari, Shchur et al., 2025; Cohen, Khwaja et al., 2025), but composing one with calibrated Bayesian uncertainty beyond a simple error model remains open, as we discuss in Section 7.

3.4. Scaling, qualitatively

NumPyro inherits JAX's transformation stack. Just-in-time compilation through `jit` produces highly optimized device code from ordinary Python model definitions. Vectorization through `vmap` lets a model defined for one unit run on a batch of units without rewriting. Parallelization across devices through `pmap` and the newer `shard_map` lets MCMC chains or independent cross-validation folds run on multiple accelerators. Switching from CPU to GPU or TPU is typically a single configuration line, `numpyro.set_platform("cuda")`, and requires no model code change.

Deterministic pseudo-random keys are a JAX design choice that pays off doubly for forecasting. Backtests become exactly reproducible across machines. Time-series cross-validation – a distinctive software challenge for forecasting – is expressed as `vmap` over a stack of cutoff dates and distinct keys rather than a Python loop over folds. Reproducibility and transparent evaluation schemes are thus properties of the substrate rather than discipline imposed after the fact. The case studies in the next section are backed by openly licensed, runnable notebooks (Orduz, 2024i), so every figure can be regenerated end-to-end. We do not report systematic benchmarks in this paper; readers interested in scaling should consult the JAX documentation and the NumPyro multi-device examples. The point we want to make is qualitative: scalability is a property of the ecosystem, not a feature one needs to opt into model by model.

Taken together, the primitives and ecosystem pieces map recurring forecasting challenges to concrete abstractions. Recursive state dynamics map to `scan`; grouped panels map to `plate`; censored and intermittent likelihoods map to composable log-probabilities inside a transition; structured covariates map to ordinary host-language functions, the capability we single out in Section 4;

reproducible backtesting maps to deterministic keys and `vmap` over evaluation folds; and linear-Gaussian substructure maps to parallelized filtering via libraries such as `cuthbert`. Forecasting problems do not merely run on the substrate. They shape which abstractions the substrate exposes and how practitioners compose them.

4. Covariate Handling: The Most Useful Capability

Before turning to the case studies, we isolate the capability that, in our experience, most often justifies reaching for a composable substrate over a packaged forecaster: full control over how covariates enter the model. A genuinely "pure" time series problem, where one only needs a black box mapping past values to future ones, is rare in practice. Far more common is a series that is business-as-usual most of the time but is reshaped by events: holidays, promotions, weather, price changes, policy interventions. The standard time series component can handle the business-as-usual dynamics, but the events demand structure.

Consider the canonical example of a holiday or promotional event. Encoding it as a single indicator column, the only option many packaged forecasters expose, discards almost everything the practitioner knows: that the effect ramps up before the day and decays after it, that it has a characteristic temporal shape, that its magnitude differs across stores or products while its shape is roughly shared. On a probabilistic substrate the practitioner instead writes the covariate effect directly: a before-and-after window, a set of Fourier terms for a periodic effect, a bump function with a learnable width, or a smooth basis whose coefficients are themselves given a prior. The covariate is a piece of model structure expressed in host-language code, not a value passed into a slot.

A particularly useful pattern, and one that recurs across panels of related series, is an event whose temporal shape is shared across the panel but whose magnitude varies from one series to the next. This is a shared basis with partially pooled, non-centered loadings, and it is a few lines of NumPyro (Listing 1): a shape per covariate that all series draw on, scaled by per-series loadings that are pooled toward a common mean. The shape borrows strength across the whole panel; the loadings let each series respond at its own amplitude.

Expressing this kind of non-time-series structure, rather than passing a covariate into a black-box slot, is in our experience the single most useful capability of a probabilistic substrate over a

```python
import jax.numpy as jnp
import numpyro
import numpyro.distributions as dist

def event_basis(X, n_nodes):
    # X[t, c, b]: value of basis function b at time t for event covariate c.
    # n_nodes: number of series in the panel.
    n_time, n_cov, n_basis = X.shape
    # Non-centered pattern: sample standardized coefficients, then shift
    # and scale them (better posterior geometry for NUTS).
    beta_loc = numpyro.sample("beta_loc", dist.Normal(0.0, 1.0))
    beta_scale = numpyro.sample("beta_scale", dist.HalfNormal(1.0))
    beta_raw = numpyro.sample("beta_raw", dist.Normal(0, 1).expand([n_cov, n_basis
        ]))
    # beta[c, b]: basis weights giving each event one temporal shape,
    # shared by every series in the panel.
    beta = beta_loc + beta_scale * beta_raw

    # Same non-centered pattern, now per series and per event.
    load_loc = numpyro.sample("load_loc", dist.Normal(0.0, 1.0).expand([n_cov]))
    load_scale = numpyro.sample("load_scale", dist.HalfNormal(1.0).expand([n_cov])
        )
    load_raw = numpyro.sample("load_raw", dist.Normal(0, 1).expand([n_nodes, n_cov
        ]))
    # loadings[j, c]: amplitude of event c for series j, partially pooled
    # toward the per-event mean load_loc[c] with spread load_scale[c].
    loadings = load_loc + load_scale * load_raw

    # Collapse the basis dimension: one temporal shape per event.
    shape = jnp.einsum("tcb,cb->tc", X, beta)
    # Scale shapes by the per-series amplitudes and sum over events:
    # the total event effect for each series at each time.
    return jnp.einsum("tc,jc->tj", shape, loadings)
```

Listing 1: A covariate-effect basis with a temporal shape shared across the panel and partially pooled, non-centered per-series loadings.

packaged forecaster. The basis is deliberately left abstract: one natural choice is a Hilbert-space Gaussian process, which we use for a nonlinear temperature response in Section 5.5, and the same machinery extends to auxiliary observations that calibrate a covariate effect against an external reference, as in Section 5.6. The case studies that follow can be read as instances of this one idea, each pairing a business-as-usual time series component with structure the practitioner controls.

5. Case Studies

This section surveys six case studies. Each subsection states the forecasting problem, the modeling idea, and what the substrate enables that off-the-shelf libraries do not. All code is publicly available through the cited blog posts and notebooks.

5.1. Hierarchical pooling for sparse panels

A canonical illustration uses the Australian tourism dataset, where each region's monthly visitor count exhibits regional level, regional trend, and seasonality with substantial cross-region variation. The model in our worked example is a hierarchical exponential smoothing specification (Orduz, 2023c, 2024d): writing j for the region and m for the seasonal period, each region carries level $l_{j,t}$, trend $b_{j,t}$, and seasonal $s_{j,t}$ states that follow the Holt-Winters recursions in one-step-ahead form:

$$
\begin{aligned}
y_{j,t} &\sim \text{Normal}(l_{j,t-1} + b_{j,t-1} + s_{j,t-m},\ \sigma_j),\\
l_{j,t} &= \alpha_j(y_{j,t} - s_{j,t-m}) + (1-\alpha_j)(l_{j,t-1} + b_{j,t-1}),\\
b_{j,t} &= \beta_j(l_{j,t} - l_{j,t-1}) + (1-\beta_j)\, b_{j,t-1},\\
s_{j,t} &= \gamma_j^*(y_{j,t} - l_{j,t-1} - b_{j,t-1}) + (1-\gamma_j^*)\, s_{j,t-m}, \qquad \gamma_j^* = \gamma_j(1-\alpha_j),
\end{aligned}
$$

where α_j, β_j, and γ_j are per-region smoothing rates for level, trend, and seasonality, γ_j^* is the standard Holt-Winters adjustment, and σ_j is the observation noise scale; beyond the training window the updates freeze, so forecasts extrapolate the last level and trend linearly and recycle the last seasonal cycle. The hierarchy sits in the priors rather than in the recursions: the level rate $\alpha_j \sim \text{Beta}(1,1)$ is left unpooled; the trend rate $\beta_j \sim \text{Beta}(c_{1,g(j)}, c_{0,g(j)})$ is pooled within states, where $g(j)$ maps each region to its state and the state-level concentrations $c_{1,s}, c_{0,s}$ are drawn from Gamma distributions with globally shared, Gamma-distributed parameters; the seasonal rate $\gamma_j \sim \text{Beta}(c_1, c_0)$ shares one global pair of Gamma-distributed concentrations across all regions;

and the noise scales $\sigma_j \sim \text{HalfNormal}(\tau)$ share a global scale τ with a Gamma prior. The pooling above is centered, written directly through Beta and Gamma hyperpriors; when a hierarchy has location-scale form, as in the companion hierarchical series (Orduz, 2024d,e,f) and the per-series event loadings of Listing 1, we use non-centered reparameterization instead, an idea that has become standard practice for hierarchical Bayesian models because it yields a posterior geometry that gradient-based samplers such as NUTS navigate more easily (Gorinova, Moore & Hoffman, 2020). In NumPyro, the entire model is a `plate` over regions wrapping a `scan` over time. Figure 1 shows the posterior predictive forecast for representative regions.

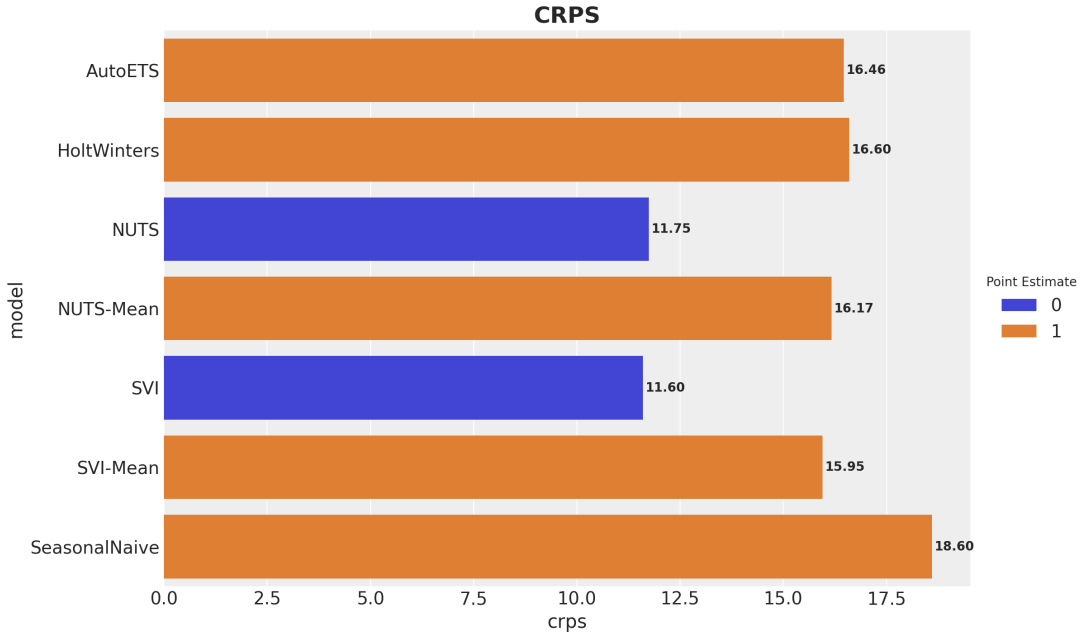

Figure 1: Hierarchical exponential smoothing on Australian tourism data. Posterior predictive medians and credible intervals are shown for several regions sharing population-level hyperpriors on smoothing rates and noise scales. Smaller regions borrow strength from larger ones without losing their distinctive seasonality.

We use "hierarchical" here in the Bayesian partial-pooling sense: unit-level parameters are drawn from population-level priors. In the forecasting literature, "hierarchical forecasting" often means forecast reconciliation, imposing coherence constraints across an aggregation structure (Athanasopoulos, Hyndman, Kourentzes & Panagiotelis, 2024); the two ideas are complementary, and reconciliation can be applied as post-processing to forecasts from a substrate model. What the substrate enables is the freedom to mix pooling levels: hierarchical in level dynamics but not in seasonality, or across regions and product categories simultaneously, each change amounting to a

few additional `plate` statements.

5.2. Intermittent demand under availability constraints

Retail demand often contains long runs of zeros. Some of those zeros reflect genuine absence of demand; others reflect stockouts where demand existed but could not be observed because the product was not on the shelf. Classical intermittent-demand methods such as Croston's method (Croston, 1972) and the TSB variant (Teunter et al., 2011) model demand as a product of an interval (or probability) component and a size component, but they do not distinguish supply-driven zeros from demand-driven zeros. Recent work has noted the importance of separating these mechanisms (Svetunkov, 2024; Pedregal & Trapero, 2024).

The availability-aware TSB model (Orduz, 2024g) modifies the TSB transition to fold a known availability indicator into the probability update. The model tracks a level z_t, an occurrence probability p_t, and an availability indicator $a_t \in \{0, 1\}$ (Teunter et al., 2011). When the product is available ($a_t = 1$), the standard TSB rules apply:

$$z_{t+1} = \alpha y_t + (1 - \alpha) z_t, \quad p_{t+1} = \beta + (1 - \beta) p_t \qquad \text{if } y_t > 0,$$
$$z_{t+1} = z_t, \quad p_{t+1} = (1 - \beta) p_t \qquad \text{if } y_t = 0.$$

When the product is unavailable ($a_t = 0$), neither state updates: $z_{t+1} = z_t$, $p_{t+1} = p_t$. The forecast is $\hat{y}_{t+1} = a_{t+1}\, z_t\, p_t$. A long stockout therefore does not corrupt the model into believing demand has vanished.

In code, this is the standard TSB recursion with the availability indicator folded into the zero branch of the probability update, and the forecast gated by availability on the way out (Listing 2).

Classical TSB is already a `scan`; the availability extension is not a new framework feature but a conditional branch in the transition function the practitioner owns. Pre-packaged intermittent-demand methods have no hook for the operational rule "freeze p_t during stockouts"; here it is one `jnp.where` inside `scan`, scalable to thousands of series via `plate` and SVI as in the cited worked example. In the cited simulation, posterior predictive forecasts recover latent demand more faithfully than a standard TSB under stockout regimes. Figure 2 shows the availability mask and the resulting forecast for a representative product.

Section 5.3 treats the same stockout reality with a censored-regression likelihood rather than a modified state update; the two approaches are complementary rather than competing.

```
from numpyro.contrib.control_flow import scan

def availability_tsb(counts, available, future=0):
    t_max, n_series = counts.shape
    with numpyro.plate("series", n_series):
        z_sm = numpyro.sample("z_sm", dist.Beta(10, 40))
        p_sm = numpyro.sample("p_sm", dist.Beta(10, 40))
        noise = numpyro.sample("noise", dist.HalfNormal(1.0))

    def transition(carry, t):
        z_prev, p_prev = carry
        pos = counts[t] > 0
        z = jnp.where(pos, z_sm * counts[t] + (1 - z_sm) * z_prev, z_prev)
        # A zero signals fading demand only if the item was on the shelf
        # (available[t] = 1); during a stockout the probability freezes.
        p = jnp.where(pos, p_sm + (1 - p_sm) * p_prev,
                      (1 - available[t] * p_sm) * p_prev)
        pred = numpyro.sample("pred", dist.Normal(z * p, noise))
        # No sales can occur while the item is off the shelf, so the
        # predicted sales are zeroed whenever available[t] = 0.
        return (z, p), available[t] * pred

    init = (counts[0], 0.5 * jnp.ones_like(counts[0]))
    scan(transition, init, jnp.arange(t_max + future))
```

Listing 2: Availability-aware TSB transition. The availability rule is one `jnp.where` branch inside the `scan` the practitioner owns. Conditioning on the observed counts and the forecast-horizon guards are elided for brevity.

5.3. Censored demand under stockouts

A complementary view of the same operational reality is censored regression (Caron, 2024; Orduz, 2023a). Consider forecasting true demand for a single SKU when historical sales are censored in two ways: an availability mask zeros out stockout periods, and a hard per-period capacity ceiling caps observed sales even when the product is on the shelf. Classical ARIMA, ETS, or Croston methods fit observed sales and therefore tend to underestimate latent demand for replenishment planning (Orduz, 2024a).

The worked example implements latent demand as an AR(2) process with Fourier seasonality

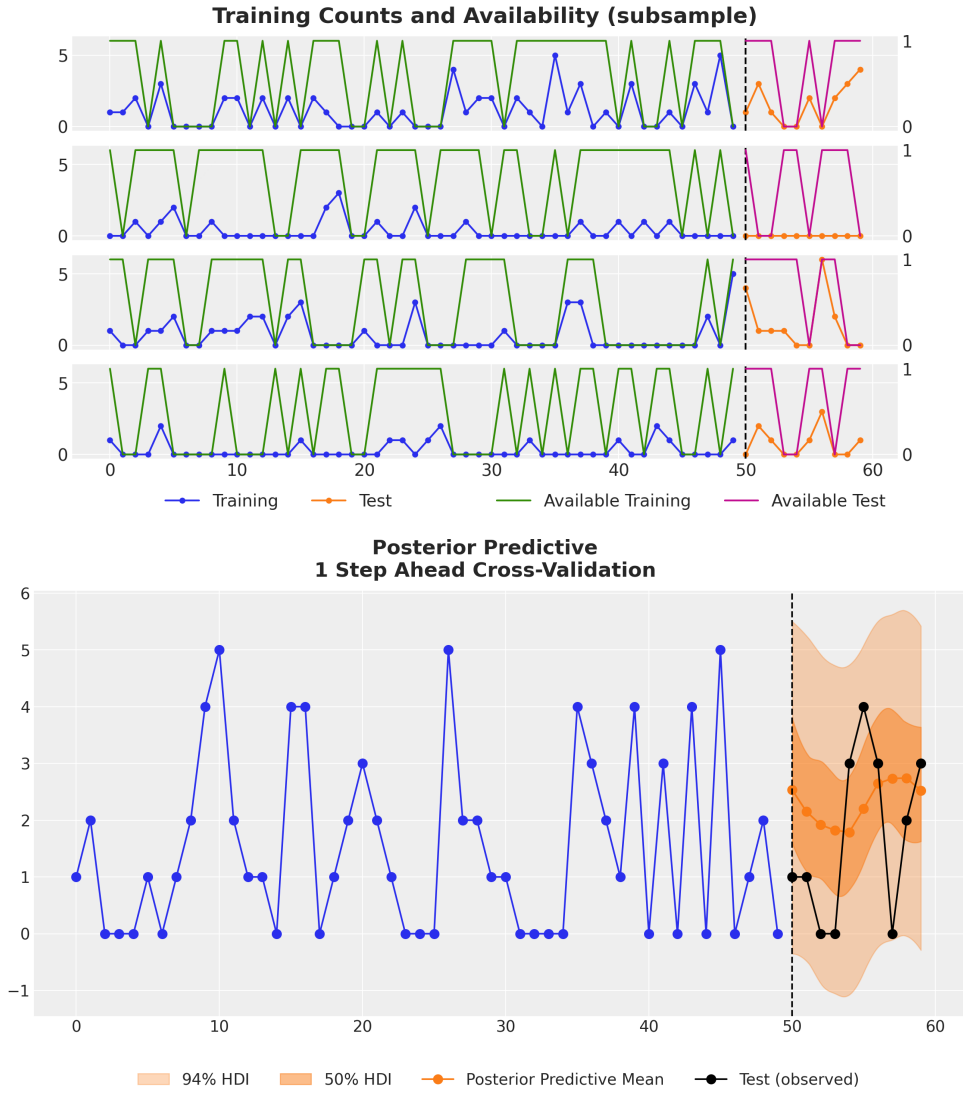

Figure 2: Top: an observed sales series with the corresponding availability mask. Long runs of zeros coincide with periods when the item was unavailable. Bottom: posterior predictive forecast from the availability-aware TSB model. The model recovers latent demand from intermittent and partially censored observations.

inside a `scan`. At each step the transition emits a predictive mean μ_t; the observation model switches between an ordinary density and a survival term depending on whether sales hit the capacity ceiling:

$$\ell(y_t \mid \mu_t, \sigma) = \log f(y_t \mid \mu_t, \sigma) \qquad \text{uncensored},$$
$$\ell(y_t \mid \mu_t, \sigma) = \log S(c_t \mid \mu_t, \sigma) \qquad \text{at capacity } c_t,$$

where f is the predictive density and S the survival function. Periods when the product is unavail-

able are handled with `handlers.mask`, which drops those steps from the likelihood so the censored observations do not distort the fit. The switch between density and survival and the AR carry are branch conditions inside the `scan` transition, with the availability mask wrapped around the whole recursion; a packaged forecaster offers no hook for this combination without forking the underlying package codebase.

Figure 3 compares a statsmodels ARIMAX baseline with the censored NumPyro posterior on a held-out horizon. The ARIMAX mean forecast (orange) stays near the range of historical observed sales; the censored posterior mean and 50%/94% highest-density intervals (green) place mass above the `max_capacity` line and track hold-out true demand more closely. Hold-out accuracy on censored sales alone can still favor the naive model, but the censored posterior is the right object for inventory planning because it forecasts the latent demand the business needs to provision against.

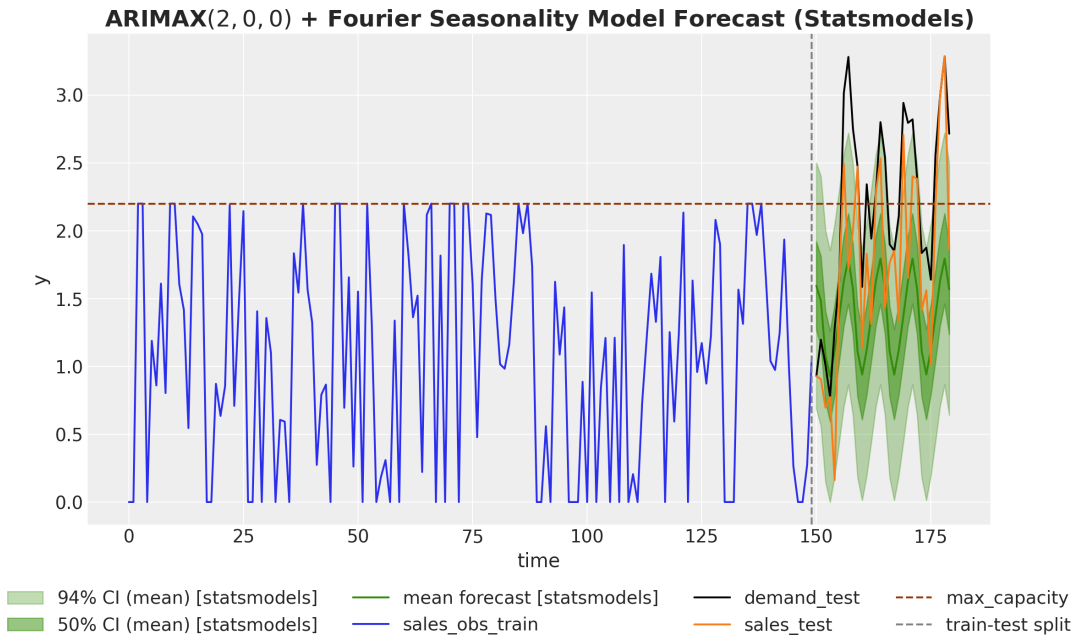

Figure 3: Censored AR(2) + Fourier forecast from Orduz (2024a). Vertical dashed line: train-test split; horizontal dashed line: `max_capacity`. Orange: statsmodels ARIMAX mean forecast. Green: censored NumPyro posterior mean and 50%/94% HDIs. Black and red: hold-out true demand and sales. A model fit to observed sales underestimates latent demand; the censored likelihood recovers it.

The same pattern extends to any domain where observations are bounded by capacity: write the appropriate log-probability inside `scan` and let inference take care of the rest.

When the data record a continuous availability signal per period rather than a binary stockout

mask, the censoring mechanism itself can be modeled: observed sales become latent demand scaled by a multiplicative availability factor. A published worked example (Orduz, 2026b) does this on FreshRetailNet-50K, a stockout-annotated dataset of 50,000 daily store-product demand series (Wang, Gu, Long, Li, Shen, Fu, Zhou & Jiang, 2025), with a hierarchical state-space model (a local level with damped trend, weekly seasonality, and pooled promotion, discount, calendar, and launch effects by store) in which a saturating function of the day's sales-weighted availability multiplies both the predictive mean and the noise scale; the factor has a learned floor that absorbs the roughly 15% of fully flagged stockout days that still record sales, and it is anchored to equal one at full availability, so the demand component reads directly as demand at full stock and the factor's functional form is one more structured covariate effect the practitioner owns in the sense of Section 4. Scenario forecasting then reduces to a covariate edit: pinning availability to one over the horizon and rerunning the forecast with the same posterior draws and the same deterministic key, a common-random-numbers comparison, converts the sales forecast into an uncensored demand forecast for replenishment. Figure 4 shows both forecasts for a series whose availability collapses late in the test window.

The example is written against `numpyro_forecast` rather than raw NumPyro, and the panel-level payoff is material: on the full 50,000-series panel the availability correction adds about 8.7% to forecast test-window volume, with 82% of the series gaining more than 1%. It also supplies a scaling data point rather than a benchmark: the end-to-end notebook runs in approximately ten minutes on a single GPU via Modal. What makes the panel fit on one accelerator is chunked sampling with host offload: posterior and posterior predictive draws are generated in fixed-size chunks, each moved off the accelerator to host memory before the next is drawn (the `batch_size` and `device="host"` arguments on the prediction path, mirrored as `predictive_batch_size` and `predictive_device` on the export path), so device memory is bounded by a single chunk rather than by the full draw array.

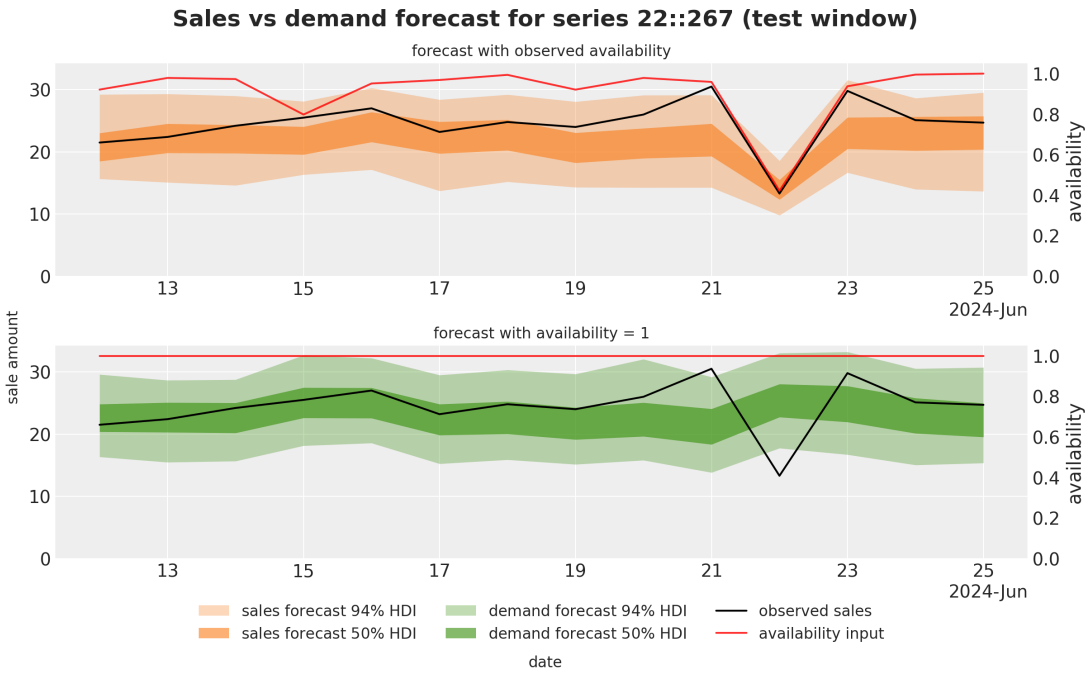

Figure 4: Sales and demand forecasts for FreshRetailNet-50K series `22::267` over the 14-day test window, from Orduz (2026b). Top: sales forecast conditioned on observed availability (orange: 50%/94% HDIs). Bottom: demand forecast with availability pinned to one (green: 50%/94% HDIs). Black: observed sales; red: availability input on the secondary axis. On the worst day (2024-06-22, availability 0.42) expected demand is 25.0 units against expected sales of 13.9, and the demand forecast carries 6.9% more volume over the window: the gap between the rows is the model's estimate of demand censored by stockouts.

5.4. Bayesian vector autoregression

Our worked example (Orduz, 2023b) fits a Bayesian VAR(p) with NUTS, where the observation vector y_t collects the k series:

$$y_t = c + \sum_{l=1}^{p} \Phi_l \, y_{t-l} + \varepsilon_t, \qquad \varepsilon_t \sim \text{Normal}(0, \Sigma), \qquad \Sigma = \text{diag}(\sigma)\, \Omega \, \text{diag}(\sigma),$$

$$c_i \sim \text{Normal}(0, 1), \quad (\Phi_l)_{ij} \sim \text{Normal}(0, 10), \quad \sigma_i \sim \text{HalfNormal}(1), \quad \Omega \sim \text{LKJ}(1),$$

where σ collects per-series scales and the correlation matrix Ω has a uniform LKJ prior, sampled through its Cholesky factor (`LKJCholesky`); the fitted example uses $p = 2$. Compared with a packaged VAR routine, what the substrate adds is direct control over this prior structure: shrinkage on the coefficient matrices and an LKJ prior on the innovation correlations are a few lines rather than a fork of someone else's estimator, and they yield full posteriors over derived quantities instead

of point estimates. There is a trade-off worth stating: for a dense linear-Gaussian state-space model like this one, marginalizing the latent states with a Kalman filter can be more efficient than sampling them through `scan`, and JAX state-space libraries such as `cuthbert` (Duffield et al., 2026) and `dynestyx` (Basis Research, 2024) compose here when that route is preferred. The recursive evaluation is, again, a single `scan` whose carry stacks the p most recent observations. Once posterior samples are available, impulse-response functions are computed inside another `vmap` over posterior draws and shock sources, which yields uncertainty bands rather than point trajectories. Figure 5 reports the resulting impulse-response functions for a small economic system.

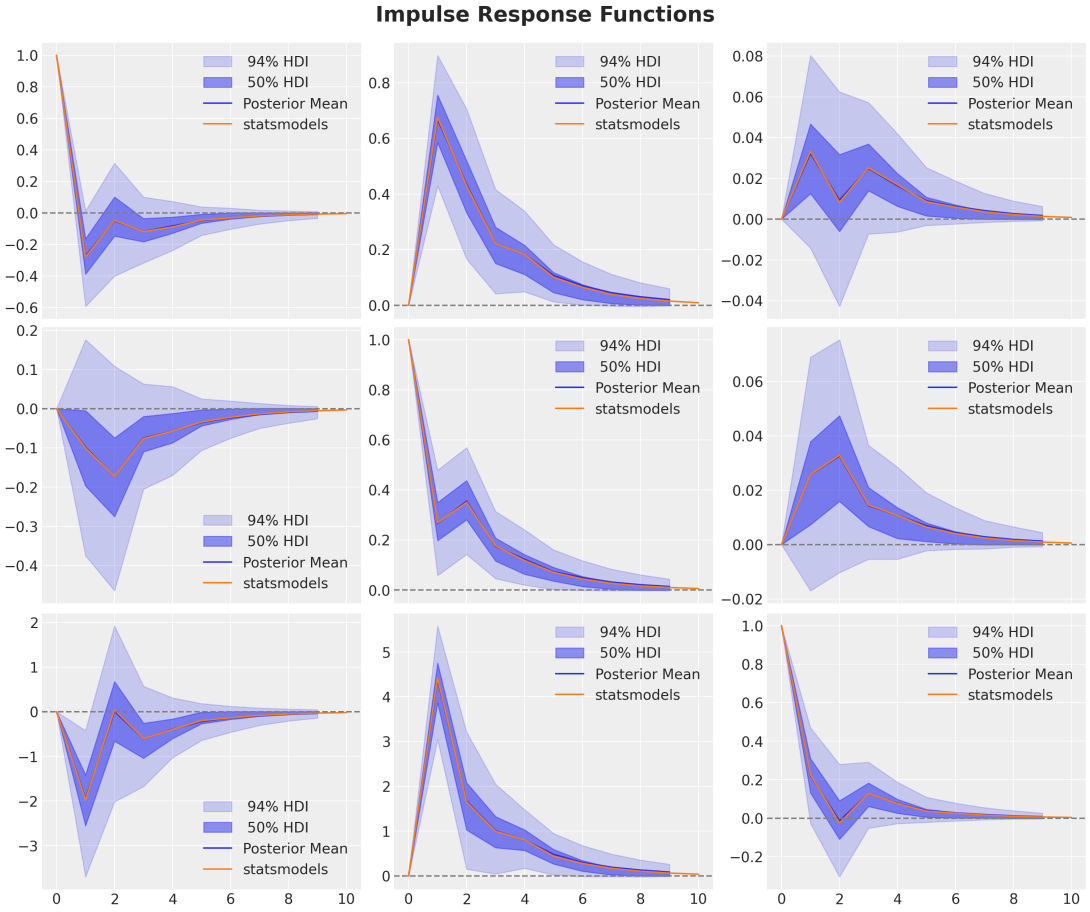

Figure 5: Posterior impulse-response functions from a Bayesian VAR(2) model fit with NUTS in NumPyro. Each panel shows the response of one variable to a unit shock in another, with shaded credible intervals reflecting parameter uncertainty.

Computing impulse-response functions across thousands of posterior samples is a vectorized JAX operation rather than a Python loop.

5.5. Gaussian processes via Hilbert-space approximation

Hilbert-space approximate Gaussian processes (HSGP) (Riutort-Mayol, Bürkner, Andersen, Solin & Vehtari, 2023) reformulate a GP as a linear-in-coefficients model that fits in linear time and composes cleanly with other components. Our electricity demand example (Orduz, 2024b) is an additive regression with heteroskedastic Student-t noise: hourly demand y_t responds to temperature T_t through a smooth varying coefficient, a temperature-dependent slope that multiplies T_t, plus zero-sum hour-of-day and day-of-week effects:

$$y_t \sim \text{StudentT}(\nu,\ \mu_t,\ \sigma\sqrt{T_t}), \qquad \mu_t = \beta_0 + \beta(T_t)\, T_t + \delta_{h(t)} + \kappa_{d(t)},$$
$$\beta(T) \approx \sum_{m=1}^{M} S_\theta(\sqrt{\lambda_m})^{1/2}\, \phi_m(T)\, w_m, \qquad w_m \sim \text{Normal}(0,1),$$

where the hour-of-day effect δ (24 components) and day-of-week effect κ (7 components) are `ZeroSumNormal` seasonal effects indexed by the hour $h(t)$ and weekday $d(t)$, (ϕ_m, λ_m) are Laplacian eigenpairs on $[-L, L]$, and S_θ is the Matérn-5/2 spectral density whose amplitude and length scale carry InverseGamma hyperpriors; the worked example uses $M = 25$ and $L = 55$ through `hsgp_matern` from `numpyro.contrib.hsgp.approximation`. We apply the HSGP to the temperature covariate here for exposition, but nothing restricts it to covariate effects: the same basis construction can represent a smooth trend or a long-period seasonal component, and these uses compose additively within one model. Because the HSGP is a linear model in its basis coefficients, it slots into the larger forecasting model with no special inference treatment. Figure 6 shows the resulting hourly demand forecast with credible bands that respect the nonlinear temperature dependence.

The HSGP is a model component, not a separate library: because it is linear in its basis coefficients, it is just one more term in the host-language model and one more choice of basis for the structured covariate effects discussed in Section 4. The temperature-response curve here is exactly the shared-basis pattern from that section, instantiated with a Gaussian-process basis.

5.6. Calibration via additional likelihoods

Forecasting practice frequently involves auxiliary information that is naturally encoded as a noisy observation rather than a prior: lift-test results, physical reference points, or expert anchors about a latent quantity. The trick, popularized by the Pyro DLM tutorial (Pyro Development Team,

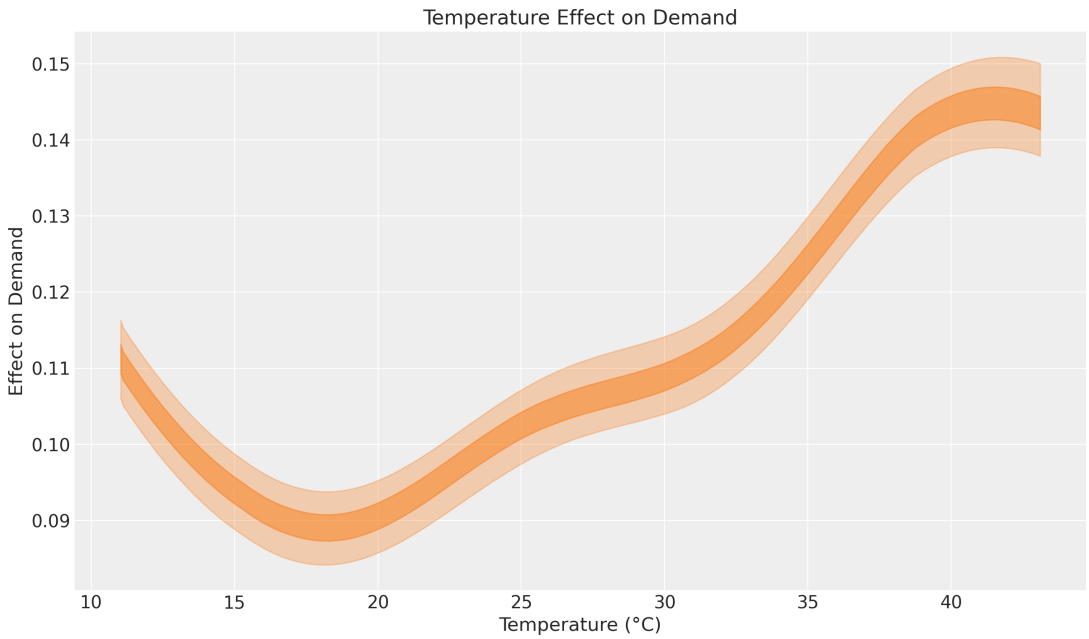

Figure 6: Electricity demand forecast from the additive model: an HSGP varying coefficient in temperature plus zero-sum hour-of-day and day-of-week effects, with heteroskedastic Student-t noise. The credible bands blend the nonlinear temperature response, the seasonal effects, and the heavy-tailed observation noise.

2020a) and explored in our electricity example (Orduz, 2024c), is to encode auxiliary information as additional likelihood terms. A lift test estimate becomes an observed-with-noise version of a posterior contrast. An expert anchor becomes an observed-with-noise version of a model output at a specific input. The forecasting model jointly explains the historical series and the auxiliary observations under a single coherent posterior. The auxiliary noise scale controls how strongly the anchor binds: log-likelihood terms add, so a small variance approaches a hard constraint, and the weighting that a penalty method would leave as a tuning parameter is here an explicit, interpretable noise assumption. Figure 7 shows how this calibration tightens the temperature-response curve in the electricity model.

The pattern uses ordinary `sample` statements with `obs=` arguments and has flowed from a Pyro tutorial through a NumPyro blog post to lift-test calibration in marketing-mix modeling (PyMC-Labs Development Team, 2024), an example of the community layer discussed in the next section.

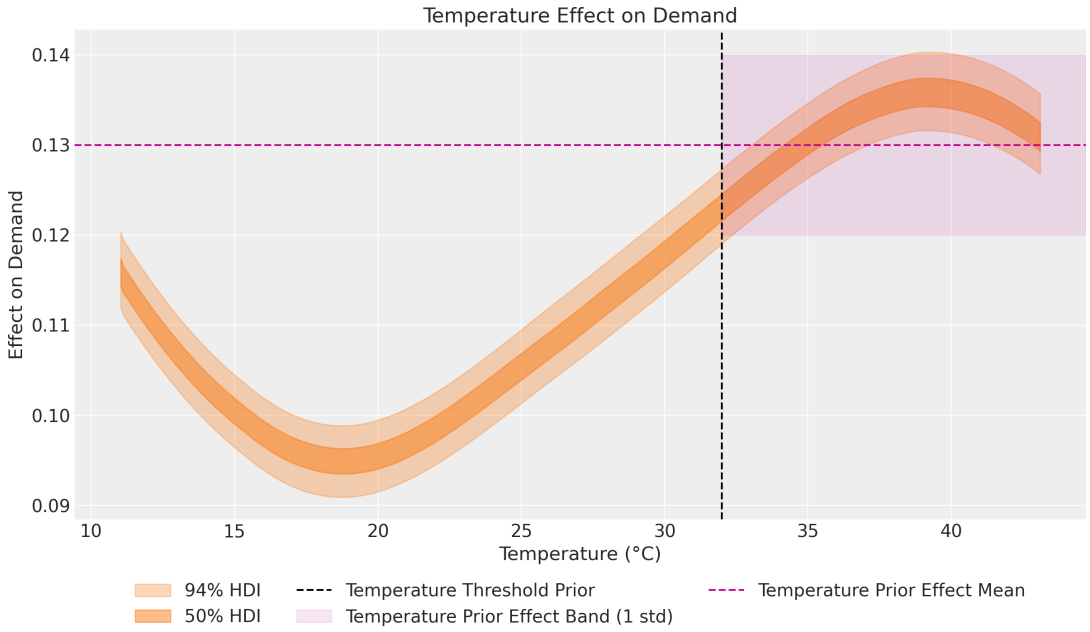

Figure 7: Calibration of the electricity demand model via an additional likelihood that anchors the temperature-response curve at an expert-elicited reference point. The calibrated posterior (right) is meaningfully tighter than the uncalibrated one (left) while remaining consistent with the historical data.

6. The Community Layer

The discussion so far has alternated between framework primitives and worked examples. That alternation is not incidental. Open-source forecasting is increasingly a two-layer story. The lower layer is composed of libraries: the substrates such as NumPyro, the pre-packaged forecasters such as the Nixtla ecosystem and sktime, the deep-learning forecasters such as GluonTS, and the classical statistical toolkits such as statsmodels (Nixtla, 2024; Löning et al., 2019; Alexandrov et al., 2020; Seabold & Perktold, 2010). The upper layer is composed of community content: tutorials, blog posts, accompanying notebooks, conference talks, forum threads, starter kits, and book-length open-access texts (Hyndman & Athanasopoulos, 2021; Svetunkov, 2023).

For a substrate-style library, the upper layer is not optional: recipes for hierarchical state-space models, availability-aware TSB transitions, censored-demand workflows, and HSGP covariate effects live in tutorials and blog posts, not the API reference. The case studies in Section 5, together with Pyro forecasting tutorials (Pyro Development Team, 2020a,b), third-party expositions (Caron, 2024; Svetunkov, 2024), and adjacent integrations (PyMC-Labs Development Team, 2024; Orduz, 2024i),

form an ecosystem of recipes that turns primitives into reproducible workflows.

The NumPyro and Pyro documentation sites carry an extensive collection of tutorials, and the blog repository underpinning many of the case studies above (Orduz, 2024i) is a living catalog of patterns. A library with strong primitives but no recipes is hard to adopt; one with excellent recipes but rigid primitives is hard to extend. Adding a clear worked example is often the highest-leverage contribution a practitioner can make.

7. Discussion and Limitations

The learning curve.. Building probabilistic forecasting models in NumPyro has a steep learning curve: JAX semantics, Bayesian modeling fundamentals, and the `scan`/`plate` style recur across use cases. There is, however, a compensating payoff. Because a NumPyro model is just JAX, almost any building block from the `jax`, `numpy`, or `scipy` world can be wired into a model, so a practitioner who hits an unusual requirement is rarely left waiting for upstream features. The community layer described in Section 6 lowers the barrier through worked examples and tutorials; this paper is intended as part of that effort.

When a turn-key library is the right answer.. A composable substrate is the right tool when the modeling assumption is the central design decision. When operational concerns dominate (friendly API, default hyperparameters, cross-validation, deployment), pre-packaged libraries such as the Nixtla ecosystem, sktime, GluonTS, and statsmodels (Nixtla, 2024; Löning et al., 2019; Alexandrov et al., 2020; Seabold & Perktold, 2010) are the better fit. Engineering teams often want a stable `fit`/`predict` surface, and wrapping a bespoke NumPyro model is extra work a packaged library provides for free. The two layers occupy complementary positions in a healthy forecasting practice.

Open problems.. Three open problems strike us as particularly relevant for the substrate-plus-community model of open-source forecasting. First, evaluation and benchmarking. There is no widely adopted benchmark suite that targets the kind of model the substrate makes natural, namely custom likelihoods, hierarchical structures, and prior calibration. The major forecasting competitions reward best-effort accuracy on standard datasets and do not directly reward the modeling flexibility that distinguishes a substrate from a library. Second, integration with large pre-trained forecasters. A simple route exists today: as noted in Section 3, released checkpoints make a frozen

forecaster callable (Ansari et al., 2025; Cohen et al., 2025), and treating its point output as the mean function of a NumPyro observation model, with an error model learned around it, is the additional-likelihood pattern of Section 5.6 applied to a foundation model. What remains genuinely open is composing the foundation model's own predictive distribution, its sampled trajectories or quantile heads, with structural components rather than discarding it: producing coherent joint trajectories across horizons and related series rather than per-step marginals, staying calibrated under distribution shift, and, when the pre-trained model instead enters as an informative prior over a latent component, avoiding double-counting the data already seen in pre-training. Promising directions include Bayesian stacking of substrate and foundation-model predictive distributions (Yao, Vehtari, Simpson & Gelman, 2018) and hierarchical error models pooled across series. Third, end-to-end probabilistic decision tooling. A posterior is only as useful as the downstream decision it supports, and the open-source landscape for translating posterior predictives into decisions under loss is far less mature than the landscape for producing the posteriors themselves.

Threats to validity.. The case studies surveyed in Section 5 are illustrative rather than exhaustive. They reflect models we have built and published in the course of practitioner-facing writing, and they are biased toward the patterns we have found useful. A different practitioner working in a different domain would surface different patterns. Our argument is therefore about plausibility and breadth: the same small set of primitives recurs across a wide variety of forecasting problems, which is evidence that the substrate is well factored, not a claim that any particular case study is the optimal solution to its underlying problem.

8. Conclusion

Open-source forecasting benefits from a layered view. Pre-packaged libraries solve standard problems behind friendly APIs. Composable probabilistic substrates such as NumPyro solve non-standard problems through programmable primitives. The community layer of tutorials and worked examples connects the two. We have argued that forecasting problems shape the abstractions a composable substrate exposes: recursive state, grouped panels, censoring, and reproducible evaluation each map to concrete software primitives and ecosystem composition patterns, and structured covariate handling, the freedom to write how events and interventions reshape a series rather than pass them into a fixed slot, is the capability that most often makes the substrate worth its learning

curve. Six case studies exercise this flexibility, and an active community of recipe writers is lowering the steep learning curve. The worked examples surveyed here are our contribution to the upper layer of the same stack.

Acknowledgments

This work stands on the shoulders of the open-source probabilistic-programming community. The authors thank the Pyro maintainers, including Eli Bingham, Fritz Obermeyer, Neeraj Pradhan, Martin Jankowiak, and many others, for the design choices and forecasting infrastructure on which NumPyro is built. The authors thank the NumPyro core team, in particular Neeraj Pradhan and Martin Jankowiak, for their ongoing maintenance and stewardship of the project. The JAX team and the broader DeepMind JAX ecosystem, including the Optax and Flax maintainers, provide the numerical foundations that make this work practical at scale. Finally, the authors thank the wider open-source forecasting and probabilistic-programming community, including the readers, commenters, issue reporters, and fellow practitioners whose feedback over many blog posts shaped the case studies surveyed in this paper. Any remaining errors are the authors' alone.

References

Alexandrov, A., Benidis, K., Bohlke-Schneider, M. et al. (2020). GluonTS: Probabilistic and neural time series modeling in Python. *Journal of Machine Learning Research*, *21*, 1–6.

Ansari, A. F., Shchur, O. et al. (2025). Chronos-2: From univariate to universal forecasting. *arXiv preprint arXiv:2510.15821*, .

Ansari, A. F., Stella, L. et al. (2024). Chronos: Learning the language of time series. *arXiv preprint arXiv:2403.07815*, .

Athanasopoulos, G., Hyndman, R. J., Kourentzes, N., & Panagiotelis, A. (2024). Forecast reconciliation: A review. *International Journal of Forecasting*, *40*, 747–767.

Basis Research (2024). dynestyx: Bayesian modeling and inference for dynamical systems built on NumPyro. `https://github.com/BasisResearch/dynestyx`.

Bingham, E., Chen, J. P., Jankowiak, M., Obermeyer, F., Pradhan, N., Karaletsos, T., Singh, R., Szerlip, P. A., Horsfall, P., & Goodman, N. D. (2019). Pyro: Deep universal probabilistic programming. *J. Mach. Learn. Res.*, *20*, 28:1–28:6. URL: `http://jmlr.org/papers/v20/18-403.html`.

Blei, D. M., Kucukelbir, A., & McAuliffe, J. D. (2017). Variational inference: A review for statisticians. *Journal of the American Statistical Association*, *112*, 859–877.

Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). JAX: composable transformations of Python+NumPy programs. `http://github.com/google/jax`.

Cabezas, A., Corenflos, A., Lao, J., Louf, R. et al. (2024). BlackJAX: Composable Bayesian inference in JAX. *arXiv preprint arXiv:2402.10797*, .

Caron, K. (2024). Modeling Anything With First Principles: Demand under extreme stockouts. `https://kylejcaron.github.io/posts/censored_demand/2024-02-06-censored-demand.html`.

CDC Center for Forecasting and Outbreak Analytics (2024). PyRenew: A Python package for Bayesian renewal modeling with JAX and NumPyro. `https://github.com/CDCgov/PyRenew`.

Cohen, B., Khwaja, E. et al. (2025). This time is different: An observability perspective on time series foundation models. *arXiv preprint arXiv:2505.14766*, .

Croston, J. D. (1972). Forecasting and stock control for intermittent demands. *Journal of the Operational Research Society*, *23*, 289–303.

Das, A., Kong, W., Sen, R., & Zhou, Y. (2023). A decoder-only foundation model for time-series forecasting. *arXiv preprint arXiv:2310.10688*, .

DeepMind, Babuschkin, I., Baumli, K., Bell, A. et al. (2020a). The DeepMind JAX Ecosystem. `http://github.com/google-deepmind`.

DeepMind, Babuschkin, I., Baumli, K., Bell, A. et al. (2020b). Optax: Composable gradient transformation library for JAX. `http://github.com/deepmind/optax`.

Duffield, S., Iqbal, S., & Corenflos, A. (2026). cuthbert: State-space model inference with JAX. `https://github.com/state-space-models/cuthbert`.

Gneiting, T., & Katzfuss, M. (2014). Probabilistic forecasting. *Annual Review of Statistics and Its Application*, *1*, 125–151.

Gneiting, T., & Raftery, A. E. (2007). Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, *102*, 359–378.

Gorinova, M. I., Moore, D., & Hoffman, M. D. (2020). Automatic reparameterisation of probabilistic programs. In *Proceedings of the 37th International Conference on Machine Learning* (pp. 3648–3657). PMLR.

Heek, J., Levskaya, A., Oliver, A., Ritter, M. et al. (2020). Flax: A neural network library and ecosystem for JAX. `http://github.com/google/flax`.

Heidemanns, M., Gelman, A., & Morris, G. E. (2020). An updated dynamic Bayesian forecasting model for the US presidential election. *Harvard Data Science Review*, *2*.

Hoffman, M. D., & Gelman, A. (2014). The No-U-Turn Sampler: Adaptively setting path lengths in Hamiltonian Monte Carlo. *Journal of Machine Learning Research*, *15*, 1593–1623.

Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice*. (3rd ed.). OTexts. URL: `https://otexts.com/fpp3/`.

Löning, M., Bagnall, A., Ganesh, S., Kazakov, V., Lines, J., & Király, F. J. (2019). sktime: A unified interface for machine learning with time series. *arXiv preprint arXiv:1909.07872*, .

Nixtla (2024). Nixtla: Open Source Time Series Forecasting. `https://github.com/Nixtla`.

NumPyro Development Team (2023a). Effect Handlers: NumPyro Tutorial. `https://num.pyro.ai/en/stable/tutorials/effect_handlers.html`.

NumPyro Development Team (2023b). Hierarchical Forecasting: NumPyro Tutorial. `https://num.pyro.ai/en/stable/tutorials/hierarchical_forecasting.html`.

NumPyro Development Team (2023c). Time Series Forecasting: NumPyro Tutorial. `https://num.pyro.ai/en/stable/tutorials/time_series_forecasting.html`.

NumPyro Development Team (2023d). Using NumPyro Models with Other Inference Libraries: NumPyro Tutorial. `https://num.pyro.ai/en/stable/tutorials/other_samplers.html`.

Obermeyer, F., Jankowiak, M. et al. (2022). Analysis of 6.4 million SARS-CoV-2 genomes identifies mutations associated with fitness. *Science*, *376*, 1327–1332. doi:`10.1126/science.abm1208`.

Orduz, J. (2023a). Bayesian Censoring Data Modeling. `https://juanitorduz.github.io/censoring/`.

Orduz, J. (2023b). Bayesian Vector Autoregressive Models in NumPyro. `https://juanitorduz.github.io/var_numpyro/`.

Orduz, J. (2023c). Hierarchical Exponential Smoothing Model. https://juanitorduz.github.io/hierarchical_exponential_smoothing/.

Orduz, J. (2023d). Notes on an ARMA(1, 1) Model with NumPyro. https://juanitorduz.github.io/arma_numpyro/.

Orduz, J. (2023e). Notes on Exponential Smoothing with NumPyro. https://juanitorduz.github.io/exponential_smoothing_numpyro/.

Orduz, J. (2024a). Demand Forecasting with Censored Likelihood. https://juanitorduz.github.io/demand/.

Orduz, J. (2024b). Electricity Demand Forecast: Dynamic Time-Series Model. https://juanitorduz.github.io/electricity_forecast/.

Orduz, J. (2024c). Electricity Demand Forecast: Dynamic Time-Series Model with Prior Calibration. https://juanitorduz.github.io/electricity_forecast_with_priors/.

Orduz, J. (2024d). From Pyro to NumPyro: Forecasting Hierarchical Models, Part I. https://juanitorduz.github.io/numpyro_hierarchical_forecasting_1/.

Orduz, J. (2024e). From Pyro to NumPyro: Forecasting Hierarchical Models, Part II. https://juanitorduz.github.io/numpyro_hierarchical_forecasting_2/.

Orduz, J. (2024f). From Pyro to NumPyro: Forecasting Hierarchical Models, Part III. https://juanitorduz.github.io/numpyro_hierarchical_forecasting_3/.

Orduz, J. (2024g). Hacking the TSB Model for Intermittent Time Series to Accommodate for Availability Constraints. https://juanitorduz.github.io/availability_tsb/.

Orduz, J. (2024h). Introduction to Stochastic Variational Inference with NumPyro. https://juanitorduz.github.io/intro_svi/.

Orduz, J. (2024i). Probabilistic Time Series Analysis: Opportunities and Applications. https://www.pymc-labs.com/blog-posts/probabilistic-forecasting.

Orduz, J. (2026a). Exponential Smoothing with NumPyro: State Space Form. https://juanitorduz.github.io/exponential_smoothing_numpyro_ssm/.

Orduz, J. (2026b). Forecasting Retail Demand Under Stockouts. https://juanitorduz.github.io/fresh_retail_stockout/.

Orduz, J. (2026c). numpyro_forecast: A NumPyro port of the Pyro forecasting module. https://github.com/juanitorduz/numpyro_forecast.

Pedregal, D. J., & Trapero, J. R. (2024). Tobit exponential smoothing. *International Journal of Forecasting*, .

Phan, D., Pradhan, N., & Jankowiak, M. (2019). Composable Effects for Flexible and Accelerated Probabilistic Programming in NumPyro. *arXiv preprint arXiv:1912.11554*, .

PyMC Developers (2024). PyMC-Extras: Statespace models module. https://github.com/pymc-devs/pymc-extras/tree/main/pymc_extras/statespace.

PyMC-Labs Development Team (2024). PyMC-Marketing: Lift Test Calibration. https://www.pymc-marketing.io/en/stable/notebooks/mmm/mmm_lift_test.html.

Pyro Development Team (2020a). Forecasting with Dynamic Linear Model (DLM). https://pyro.ai/examples/forecasting_dlm.html.

Pyro Development Team (2020b). Pyro M5 Starter Kit. https://github.com/pyro-ppl/Pyro-M5-Starter-Kit.

Riutort-Mayol, G., Bürkner, P.-C., Andersen, M. R., Solin, A., & Vehtari, A. (2023). Practical Hilbert space approximate Bayesian Gaussian processes for probabilistic programming. *Statistics and Computing*, *33*, 17. doi:10.1007/s11222-022-10167-2.

Seabold, S., & Perktold, J. (2010). statsmodels: Econometric and statistical modeling with Python. https://www.statsmodels.org/.

Smyl, S. (2020). A hybrid method of exponential smoothing and recurrent neural networks for time series forecasting. *International Journal of Forecasting*, *36*, 75–85.

Svetunkov, I. (2023). *Forecasting and Analytics with the Augmented Dynamic Adaptive Model (ADAM)*. Chapman and Hall/CRC. URL: https://openforecast.org/adam/.

Svetunkov, I. (2024). Why Zeroes Happen? https://openforecast.org/2024/11/18/why-zeroes-happen/.

TensorFlow Probability Development Team (2024). TensorFlow Probability: Structural time series (tfp.sts). https://www.tensorflow.org/probability/api_docs/python/tfp/sts.

Teunter, R. H., Syntetos, A. A., & Babai, M. Z. (2011). Intermittent demand: Linking forecasting to inventory obsolescence. *European Journal of Operational Research*, *214*, 606–615.

Vadivelu, N. et al. (2024). Awesome JAX: A curated list of JAX libraries, projects, and resources. https://github.com/n2cholas/awesome-jax.

Wang, Y., Gu, J., Long, L., Li, X., Shen, L., Fu, Z., Zhou, X., & Jiang, X. (2025). FreshRetailNet-50K: A stockout-annotated censored demand dataset for latent demand recovery and forecasting in fresh retail. *arXiv preprint arXiv:2505.16319*, .

Wong, K. W. K., Gabrié, M., & Foreman-Mackey, D. (2023). flowMC: Normalizing flow enhanced sampling package for probabilistic inference in JAX. *Journal of Open Source Software*, *8*, 5021. doi:10.21105/joss.05021.

Yao, Y., Vehtari, A., Simpson, D., & Gelman, A. (2018). Using stacking to average Bayesian predictive distributions (with discussion). *Bayesian Analysis*, *13*, 917–1007.